



PROJET SCIENTIFIQUE COLLECTIF

**GÉNÉRATION DE PNJ PAR L'INTELLIGENCE
ARTIFICIELLE**

2023 - 2024

ABBOUD Zakaria
MERCIER Gabriel
TANG Yingwei
TOUAYOUCH Brahim
ZOU Siyuan

TABLE DES MATIÈRES

1	Introduction	3
2	Modélisation du Back-end	4
2.1	Modélisation de l'environnement	4
2.2	Modélisation de la mémoire	5
2.2.1	Structure	5
2.2.2	Mémoire court-terme	6
2.2.3	Mémoire épisodique	7
2.2.4	Conditions Initiales	9
2.2.5	Personnalité / Émotions / Relations	9
2.3	Modélisation de l'action/Interaction	10
2.3.1	Modélisation du plan d'un Agent	10
2.4	Exécution d'une tâche	12
2.5	Gestion de l'interaction entre agents	13
2.5.1	Modélisation et implémentation de l'interaction entre agents :	13
2.5.2	Décalage du plan lorsqu'il y a une interaction	15
2.6	Réflexion	16
3	Modélisation du Front-end	17
3.1	Utilisation de Tiled et de la bibliothèque Pygame	17
3.2	Connexion Back-end -> front-end	18
4	Historique et points clefs	20
5	Difficultés rencontrées	22
5.1	Modélisation et Implémentation de la mémoire	22
5.2	Le choix des prompts	23
5.3	Exécution en parallèle	24
5.4	Gestion du temps et Optimisation des scripts	25
6	Organisation du travail	26
7	Degré de réalisation des Objectifs	27
7.1	Objectifs	27
7.2	Degré de réalisation des objectifs	27
8	Conclusion	30
9	Bibliographie	32
10	Annexes	32

1

INTRODUCTION

La possibilité d'appliquer l'Intelligence Artificielle (IA) pour générer des personnages non joueurs plus réalistes et intelligents est une direction innovante révolutionnant la manière dont les joueurs interagissent avec des mondes virtuels. En effet, depuis peu, l'industrie du jeu vidéo tend à utiliser l'IA afin de créer des Personnages Non Joueurs (PNJ) plus réactifs, autonomes et réalistes, renforçant ainsi l'immersion et offrant une expérience personnalisée. Récemment, l'université de Stanford a mené une recherche (Park et al., 2023) visant à intégrer le modèle de LLM GPT-3.5-turbo de OpenAI dans la gestion de PNJ. Les PNJ deviennent bien plus que de simples agents préprogrammés mais évoluent et s'adaptent en temps réel en fonction à la fois des choix et des actions des autres mais également de leur propre expérience. Ils peuvent réagir de manière réaliste, prendre des décisions, et même entretenir des dialogues naturels avec les joueurs, afin de simuler le comportement humain. Inspiré du projet de Stanford, notre projet a pour but de créer un petit village virtuel avec quelques agents gérés par GPT-3.5-turbo qui sert de jeu d'interaction et d'une simulation d'une vie communautaire humaine. Nous séparons le système du jeu en deux parties : le back-end et le front-end. D'une part, utilisant Tiled et le module pygame, le front-end constitue une interface d'image et la représentation visuelle du jeu. D'autre part, le back-end construit un système de la mémoire, de plan et de prise de décision simulant le comportement du point de vue neuroscientifique.

En utilisant les principes du *prompt engineering*, les agents ont réussi à développer la capacité d'agir et d'interagir avec d'autres agents du village. Le modèle GPT-3.5-turbo permet de produire du texte naturel, enrichissant ainsi la mémoire de l'agent. Cette mémoire est exploitée pour prendre des décisions et orchestrer les interactions entre les agents. La conception d'un mécanisme de mémoire à la fois simple et suffisamment complexe pour conférer un aspect humain à l'agent constituait l'un des aspects cruciaux de notre projet. Grâce à des essais et à l'exécution de simulations, nous avons pu améliorer continuellement le projet, notamment en affinant les prompts pour les adapter à chaque tâche.

2

MODÉLISATION DU BACK-END

2.1 MODÉLISATION DE L'ENVIRONNEMENT

L'environnement a été conçu en utilisant une classe Python qui prend en compte à la fois la structure de la carte et la gestion du temps. Nous avons choisi de diviser la carte en trois niveaux distincts : les bâtiments, les pièces ou zones, et les objets. Cette structure permet de créer des adresses facilement accessibles par les agents, formulées sous la forme *building :hall :object*.

La classe "Environment" contient un attribut crucial appelé "case_details". Cet attribut décrit, pour chaque position (x, y) sur la carte, les événements en cours, les agents présents à cet endroit, ainsi que l'adresse correspondante au format *building :hall :object*. Cette modélisation nous a permis d'établir efficacement un lien entre le langage naturel utilisé pour donner l'adresse (généré par le modèle de LLM) et les coordonnées réelles sur la carte.

De plus, cette conception assure une communication efficace entre les agents. Elle leur permet d'interagir entre eux, de modifier le contenu de l'attribut "case_details", et de suivre toutes les actions effectuées dans l'environnement.



FIGURE 1 – Image extraite de l'environnement du jeu et montrant sa décomposition

En outre, la carte intègre une méthode très pratique nommée "get_path". Cette fonction utilise l'algorithme A* pour déterminer le chemin optimal entre chaque point de la carte. Elle facilite ainsi la navigation des PNG, permettant des déplacements intelligents et efficaces dans l'environnement simulé.

2.2 MODÉLISATION DE LA MÉMOIRE

2.2.1 • STRUCTURE

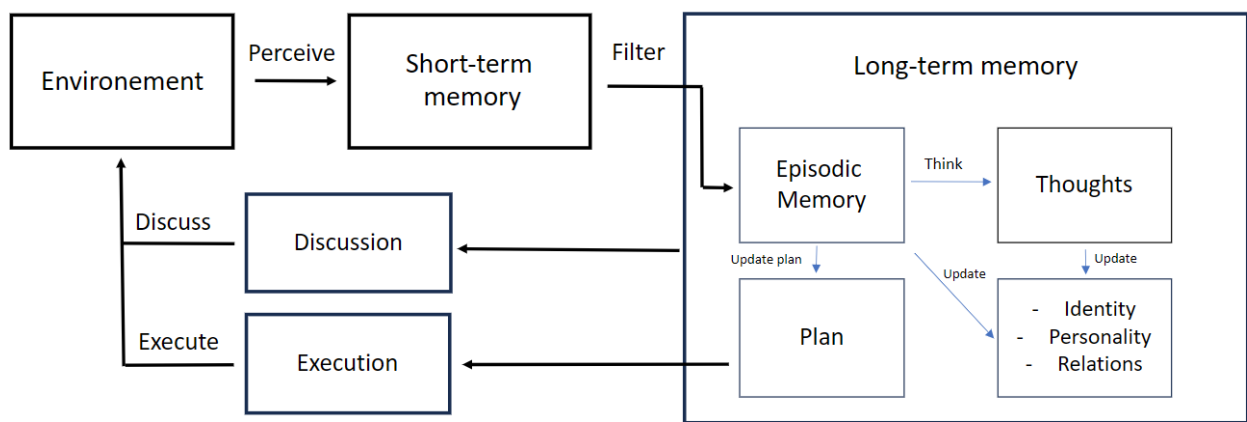


FIGURE 2 – Structure globale de la mémoire

La structure de la mémoire de nos agents s'appuie à la fois sur les neurosciences, le projet de Stanford et nos diverses réflexions. Nous avons ensuite adapté cela pour prendre en considération la possibilité de concrétiser les notions abstraites de la mémoire sous forme d'une implémentation en Python et selon nos capacités (en terme de compétences et de temps alloué).

D'après le modèle d'Atkinson-Shiffrin (Atkinson & Shiffrin, 1968), la mémoire est divisée en trois sous-systèmes : la mémoire sensorielle (*sensory memory*), la mémoire court-terme (*short-term memory*) et la mémoire long-terme ou épisodique (*episodic memory*). Les informations perçues de l'environnement sont instantanément insérées dans la mémoire sensorielle. Les informations y restent pour un temps très court avant d'entrer dans la mémoire à court-terme après un premier filtrage. Ensuite, après un second tri plus personnel, les informations sont insérées dans la mémoire long-terme. Dans notre projet, comme l'environnement des agents n'est pas très grand et détaillé, la majorité des informations perçues sont importantes (dans l'expérience réelle, un grain de sable sur le bord de la route est vite oublié par exemple). Par conséquent, nous nous passons de la mémoire sensorielle, et nous conservons la mémoire à court-terme qui effectue un tri personnel à chaque agent.

Dans le modèle défini par la neurosciences (voir ci-dessus), la mémoire long-terme est complexe et possède de nombreuses fonctions. Nous avons fait le choix de la simplifier au maximum

en la dénaturant le moins possible. Scientifiquement, elle est séparée principalement en deux parties : la mémoire procédurale et la mémoire épisodique. La première est responsable du stockage et de l'exécution des savoir-faire et des compétences. Dans notre projet elle est directement simulée par GPT-3.5-turbo, donc elle n'apparaît pas dans notre code. La deuxième (mémoire épisodique) consiste en des souvenirs spécifiques d'événements et de situations. Dans notre projet elle est simulée par un fichier json stockant chaque événement. La mémoire épisodique est la base de l'induction des pensées de haut niveau et de la décision pour générer le plan.

Les différentes parties et fonctions mentionnées ci-dessus sont explicitées plus en détail dans la suite du rapport.

2.2.2 • MÉMOIRE COURT-TERME

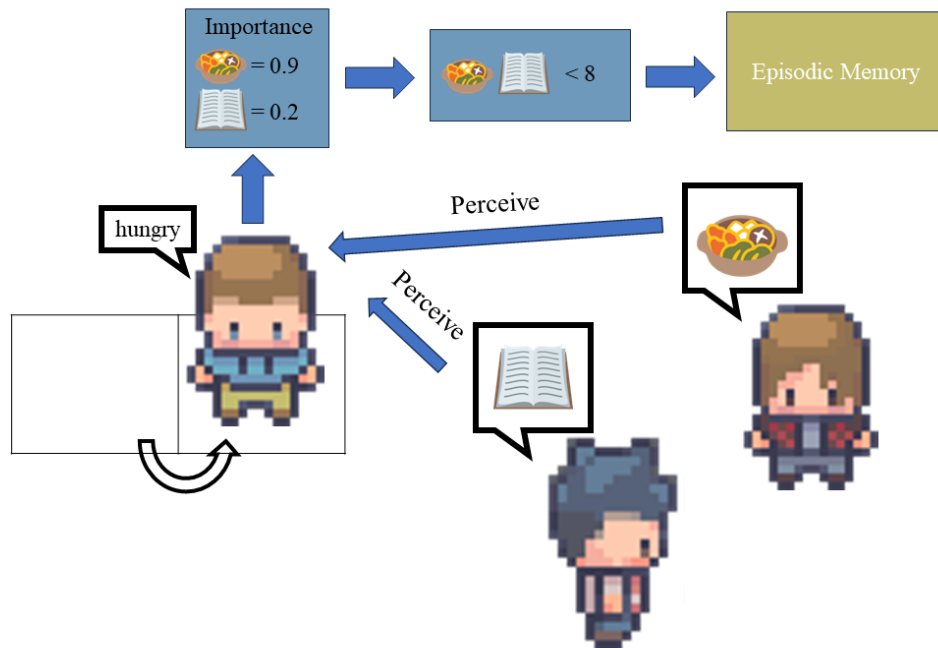


FIGURE 3 – Tri de la mémoire

Comme décrit précédemment, la mémoire court-terme décrit l'étape entre la perception directe de l'environnement et son intégration dans la mémoire épisodique. À l'image du fonctionnement réel du traitement humain des données, notre agent perçoit. Pour cela nous avons implémenté la fonction "perceive" qui détecte, dans notre cas, les actions des autres joueurs se déroulant dans les cases adjacentes à la leur, au sein d'une même pièce, d'une même zone (appelé "hall" dans le projet), puis renvoie la liste des agents proches. À chaque pas, l'agent perçoit et chaque événement perçu est ajouté à un set "seen". À chaque fin de mouvement (pour des questions de complexité), le set est trié : en effet, GPT-3.5-turbo associe à chaque événement une

importance (personnelle à chaque agent : par exemple, se faire offrir une jeu d'échecs peut avoir une importance différente entre deux personnes). Ensuite, seules les 8 plus importantes (d'après des limitations physiologiques étudiées par les neurosciences (Posner, 1966)) sont conservées et directement intégrées à la mémoire épisodique. On s'assure de ne pas ajouter deux fois le même évènement mais en augmentant éventuellement son importance. Enfin, le set est réinitialisé. Cette perception s'effectue continuellement lorsque l'action est en cours (hors déplacement) et les évènements perçus sont triés à l'issu du mouvement de la tâche suivante. L'activité constante de cette méthode nous servira énormément pour la détection des éventuelles interactions entre agents.

2.2.3 • MÉMOIRE ÉPISODIQUE

La composante principale de la mémoire n'est autre que la mémoire épisodique, qui stocke les événements factuels les plus importants que l'agent a vécus ou rencontrés, elle est organisée et stockée dans des fichiers JSON pour chaque PNJ, comprenant des événements, des pensées et des discussions filtrés par la mémoire à court terme. Chaque fichier JSON contient un dictionnaire qui associe ces éléments à leur vecteur d'*embedding* correspondant, permettant ainsi une représentation numérique des informations : ces *embeddings* sont calculés en transformant les phrases en vecteurs selon le modèle de NLP Bert (Devlin et al., 2018), cette modélisation nous permettrait d'utiliser et d'extraire les données pertinentes de la mémoire en fonction des besoins du PNJ (voir explication ci-dessous). De plus, chaque entrée dans ce dictionnaire est accompagnée de l'heure à laquelle l'événement a été enregistré et de son score d'importance pour évaluer la pertinence de l'information. Voici un exemple de la mémoire épisodique d'Elsa une des PNJ :

```
{
  "embeddings": {
    "Elsa is Stretching and light exercise": [
      0.0032687257044017315,
      0.025652652606368065,
      0.10087814182043076,
      ....
    ]
  },
  "nodes" : [
    [
      "07:35 AM",
      "Elsa is Stretching and light exercise",
      6
    ],
    [
      "07:59 AM",
      "Elsa is Preparing a healthy breakfast",
      7
    ]
  ],
}
```

```

    ....
  ]
}

```

Cette mémoire est accompagnée d'une fonction "retrieve" pour extraire des informations pertinentes de la mémoire. Le système convertit une phrase donnée en un vecteur d'embedding, puis calcule le cosinus de l'angle entre ce vecteur et ceux de la mémoire. Le score de similarité cosinus le plus élevé indique la correspondance sémantique la plus étroite entre la phrase et les événements stockés dans la mémoire de l'agent. Les éléments de mémoire sont donc triés par rapport à ce score et sont extraits pour servir lors d'une interaction, d'une discussion ou d'une réflexion afin d'obtenir une liste d'événements proches au niveau sémantique.

Pour visualiser la mémoire épisodique, nous avons opté pour la bibliothèque Python NetworkX, qui est très optimisée en termes de complexité et également très intuitive à utiliser pour gérer les graphes. La mémoire épisodique est alors convertie en un graphe où le lien entre chaque nœud du graphe est représenté par le score de similarité cosinus entre les deux nœuds. Pour une meilleure visualisation des liens, nous avons introduit une valeur seuil qui décide si deux nœuds doivent être reliés ou non. Cette valeur seuil est ajustée en fonction de la taille du graphe pour permettre une visualisation plus claire.

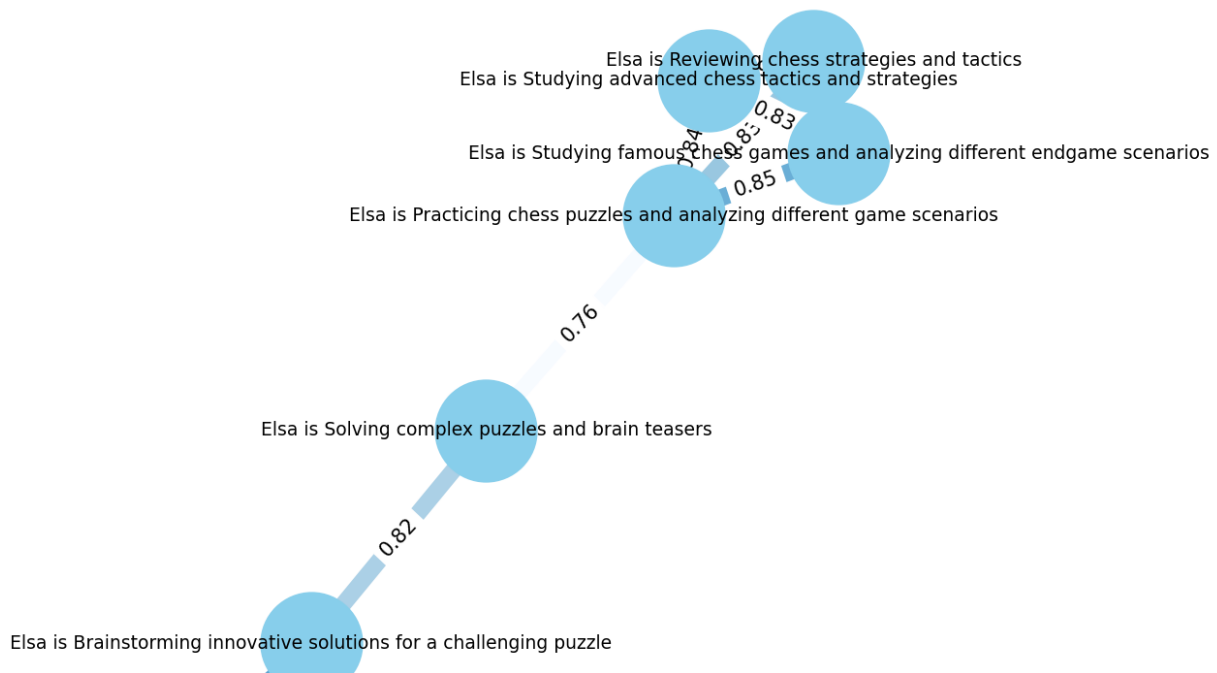


FIGURE 4 – Exemple de la partie de la mémoire épisodique d'Elsa liée au jeu d'échec et quelques événements proches

2.2.4 • CONDITIONS INITIALES

L'objectif de notre travail est de générer de l'émergence d'un comportement humain chez des PNJ. Or, tout système d'émergence contient des règles, et des informations initiales connues du système. En effet, dans notre simulation, les agents sont comme posés dans une temporalité qui suppose qu'ils possèdent un passé (ils ne commencent pas en étant des nouveaux nés) ayant marqué leur histoire et donc à priori leur futur. Dans notre cas, il s'agit de définir en amont un certain nombre de caractéristiques assurant une plus grande profondeur et personnalisation des actions de nos agents. Parmi ces caractéristiques, nous comptons : le nom, l'âge, le profil, les motivations intrinsèques, valeurs et croyances, préférences et rejets, talents et compétences, histoire personnel. Ces caractéristiques, sous forme de phrases (sauf l'âge) se sont ajoutées au fur et à mesure du projet selon le besoin de pertinence, de créativité et de profondeur à l'instant t .

2.2.5 • PERSONNALITÉ / ÉMOTIONS / RELATIONS

Nous avons implémenté la gestion de la personnalité, de l'émotion à l'aide de classes dédiées. Chaque agent possède donc un objet émotion, personnalité (les relations sont traitées à part uniquement via des fonctions). Stockées dans des fichiers json ("current_emotion.json", "personality.json" et "relationship.json") accessibles à tous, elles sont caractérisées par des tableaux où chaque indice représente une caractéristique. Par exemple pour les émotions, chaque indice correspond, dans l'ordre d'apparition à : peur, surprise, tristesse, dégoût, colère, anticipation, joie et acceptation basées sur la roue des émotions de Robert Plutchik (Plutchik, 1980) (qui a le mérite d'être simple à comprendre et à interpréter pour un LLM) et comme valeurs des nombres entre 0 et 10 : 0 signifiant l'absence, 10 la prédominance. La personnalité quant à elle est basée sur le modèle OCEAN (neuroticisme, extraversion, ouverture, agréabilité, conscience). Pour les relations, il s'agit de : domination, appréciation, solidarité, familiarité (Bickmore et al., 2005). Ces valeurs sont entrées manuellement en début de simulation selon le type de personnalité, d'état émotionnel et de relations que nous souhaitons pour nos agents (conditions initiales).

Une des fonctions principales (utiles en simulation) de ces classes est celle de récupération des valeurs sous forme de string ("print_emotions" par exemple) permettant d'être directement utilisées dans des prompt d'autres fonctions. La seconde fonction importante de ces classes est celle de mise à jour des valeurs ("update_emotion"). En effet, à l'issue d'un événement extérieur ou non, la valeur de l'émotion peut être mise à jour (d'après le modèle OCC (Ortony et al., 1988) : GPT-3.5-turbo décide (selon l'événement, la personnalité et les éléments de mémoire pertinents, c'est à dire proches sémantiquement) si l'état émotionnel doit changer et si oui vers quelles valeurs. Techniquement, cette mise à jour s'effectue si l'importance de l'événement dépasse un seuil défini arbitrairement (de sorte à avoir environ 5 mise à jour émotionnelle par jour). Dans le cas de la personnalité, cette mise à jour est bien moins régulière (journalière) et moins significative (la variation maximale par caractéristique est majorée).

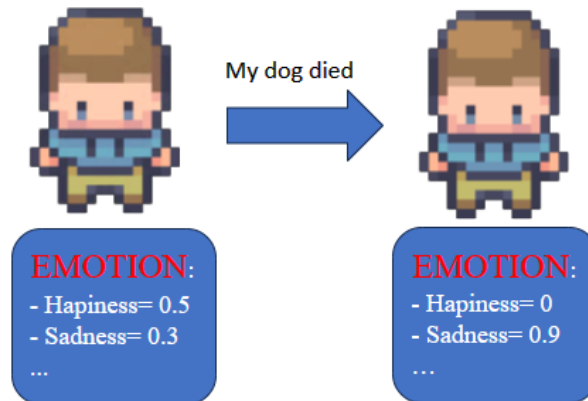


FIGURE 5 – Structure des émotions et mise à jour

2.3 MODÉLISATION DE L’ACTION/INTERACTION

2.3.1 • MODÉLISATION DU PLAN D’UN AGENT

Modéliser et structurer le plan d’un agent est une partie très importante de notre travail. Pour résoudre cette question, nous avons proposé une structure simple, intuitive, bien organisée, tout en étant raisonnable et logique. L’idée est de générer pour chaque agent un plan de la journée avant le commencement de la simulation. Pour générer ce plan, il faut fournir au modèle GPT-3.5-turbo des informations personnelles (les conditions initiales décrites précédemment) au sein d’un prompt qui ressemble à ceci :

You are an individual with a profile defined by the information in the text delimited by <>.

information : <info>

You want to create a plan of the day. Your response must be in JSON format with the key "plan_of_the_day" and the value is the list of tasks. Tasks should be represented as a python list of [start time task, end time task, a description of the task] as the corresponding value.

The end time of one task marks the beginning of the next task. time must have this format "xx :xx AM" or "xx :xx PM". The description must start with a gerund (verb + ing). Do not include other people in the tasks.

###

Create your plan of the day starting from the moment you wake up to when you go to sleep.

Voici un exemple des informations d’un agent qui représentent son identité (conditions initiales décrites ci-dessus) :

```

"Elsa": {
  "name": "Elsa",
  "age": 23,
  "lifestyle": "Elsa goes to bed at 11pm, awakes up at 7am.",
  "profile": "Elsa is an engineer at Orange. She is married and...",
  "initial_position": {
    "x": 8,
    "y": 54
  },
  "Goals and motivations": "Elsa aims to lead a major engineering...",
  "Values and beliefs": "She values creativity and innovation in...",
  "Preferences and dislikes": "Elsa enjoys classical music, has a...",
  "Skills and talents": "Beyond her engineering skills, Elsa is a...",
  "Personal history": "She was raised in a supportive environment..."
}

```

Ainsi, le modèle produit un plan au format JSON qui inclut plusieurs tâches. Chaque tâche est composée de quatre éléments : le début de la tâche, la fin de la tâche, ainsi que la description de la tâche sous forme de phrase. Voici un extrait du plan généré pour l'agent mentionné précédemment :

```

[
  [
    "7:00 AM",
    "7:30 AM",
    "Elsa's morning routine, including stretching and breakfast preparation"
  ],
  [
    "7:30 AM",
    "8:30 AM",
    "Reviewing chess strategies and tactics"
  ],
  ...
]

```

Ainsi est représenté le plan initial de chaque agent. Lors de la simulation, au début de chaque tâche, celle-ci est subdivisée en sous-tâches qui peuvent être alors exécutées par l'agent. Ce mode de fonctionnement permet d'éviter de générer un plan initial trop gros qui sera a priori destiné à être modifié au cours de la journée. Par exemple, la tâche *Elsa's morning routine, including stretching and breakfast preparation* a été décomposée par le modèle en deux sous-tâches :

```
[
  [
    "7:00 AM",
    "7:10 AM",
    "Elsa's and James's House:Bedroom:Bed Elsa side",
    "Stretching and light exercise"
  ],
  [
    "7:10 AM",
    "7:30 AM",
    "Elsa's and James's House:Kitchen:Counter",
    "Preparing a healthy breakfast"
  ]
]
```

Chaque sous-tâche contient, en plus des trois attributs définis précédemment, un attribut représentant la localisation où la tâche sera effectuée. Du point de vue technique, cette sélection de localisation est effectuée (parmi une liste fournie) par GPT-3.5-turbo en 2 temps : un premier prompt pour le choix du bâtiment et de la pièce (*building :hall*), un second pour l'objet (*:object*) avec lequel il doit interagir. Cependant, cette partie présente des difficultés (voir partie difficultés). Voici le prompt utilisé pour accéder à l'objet :

*You are an individual named #name# who is supposed to do the following task :
#task#, in the following localisation : #localisation#. Your response must be one
word. It must be from the list given.

Choose the most appropriate object to do your task from the following options :
#objects#.*

2.4 EXÉCUTION D'UNE TACHE

À partir de chaque sous-tâche, le PNJ déduit le chemin qu'il doit suivre pour atteindre l'emplacement où la sous-tâche doit être accomplie, en utilisant la fonction "find_path" (à l'aide de "get_path" mentionné ci-dessus). Tout au long de l'exécution de ces actions, l'environnement se met à jour dynamiquement en fonction des modifications induites par les actions des PNJ. En effet, l'exécution d'une sous-tâche se découpe en plusieurs étapes : l'agent se déplace (si nécessaire) jusqu'au lieu prévu. A chaque pas, l'événement en cours est ajouté à l'environnement. Chaque PNJ agit de manière autonome en suivant les étapes déduites de la tâche assignée, contribuant ainsi à l'évolution constante de l'état de l'environnement. Cela permet alors aux autres PNJ d'apercevoir cette évolution et de réagir si nécessaire. La perception d'autrui se fait donc via l'environnement, à l'image du fonctionnement réel de la perception.

2.5 GESTION DE L'INTERACTION ENTRE AGENTS

2.5.1 • MODÉLISATION ET IMPLÉMENTATION DE L'INTERACTION ENTRE AGENTS :

Il est illogique de penser à simuler la vie humaine collective en créant uniquement des agents capables de planifier leur journée et d'exécuter leur plan de la manière la plus humaine possible. En effet, la dimension de l'interaction et l'influence mutuelle entre les agents est sans doute celle qu'il ne faut surtout pas négliger. Ceci nécessite des changements dans la structure de modélisation, en prenant en compte l'interaction entre les agents. Tout au long de notre projet, nous avons supposé que la seule forme d'interaction possible entre les agents était une éventuelle discussion. La première question que nous avons dû résoudre avant d'entamer la modélisation de l'interaction est de savoir quand deux agents ou plus vont discuter ensemble. Dans une première modélisation, nous nous sommes basés sur un critère purement spatial, à savoir que deux agents sont susceptibles d'interagir s'ils sont suffisamment proches l'un de l'autre. Bien sûr, cette approche très restrictive est loin d'être réaliste, car les interactions humaines peuvent être plus variées et peuvent avoir lieu à des distances non fixes (par exemple, à distance). Cependant, notre objectif étant de créer des agents autonomes vivant dans un village aussi simple que possible pour observer leurs comportements, ce choix de critère pour déclencher une discussion nous a semblé pertinent pour capturer de nombreux comportements humains influencés par les interactions avec les autres. De plus, il est clair que prendre en compte toutes les conditions réelles est un projet très ambitieux qui nécessiterait plus d'expertise.

D'un point de vue technique, puisque tous les agents disposent de la méthode "perceive" pour observer leur environnement, nous avons modifié cette fonction. Elle alimente désormais la mémoire sensorielle en détectant les événements les plus proches et pertinents, tout en surveillant les agents à proximité. Ainsi, grâce à la méthode "perceive" constamment active, un agent peut surveiller en permanence les agents à proximité. De plus, pour des raisons techniques liées à la parallélisation des agents (discutée ultérieurement), nous avons restreint les interactions entre les agents se croisant lorsque les agents en question sont en mouvement. Même si ce n'est pas toujours le cas dans la vie réelle, cela économise beaucoup de temps d'exécution et évite les problèmes liés à la parallélisation. Lorsque deux agents se croisent, la méthode "interact" est déclenchée par celui qui voit l'autre en premier. Cette méthode vise à déterminer si les agents entameront une discussion. Pour ce faire, le modèle évalue un prompt contenant les informations actuelles des agents, leur relation et leur état de mémoire. La réponse peut être 'NO' en cas de refus de discussion ou le nom de l'agent avec lequel discuter si la discussion est acceptée. Cette décision est basée sur un prompt conçu pour inclure les informations essentielles à la prise de décision concernant la discussion. Le prompt retenu est le suivant :

As an individual, you are walking to the location where you will carry out the event #task_of_the_agent#. In your way you meet the people in the following list #other_agents_info#. Each sublist includes the person's name, your personal relationship with the person, and the event they are performing. Respond with the name

of the person you wish to interact with or respond with 'NO' if you prefer not to engage.

Important : you do not need to stick to your schedule. Your answer must be only one word.

###

Respond with THE NAME OF THE PERSON or 'NO'.

Pour garantir le déroulement fluide de cette interaction potentielle et éviter que les agents ne partent trop tôt, ceux impliqués feront une brève pause en attendant l'éventuelle discussion. Même s'ils ne dialoguent pas à la fin, ils échangeront des salutations. Si la méthode "interact" renvoie le nom de l'agent avec lequel discuter, celui qui a lancé l'interaction informera l'autre agent via le fichier JSON *discussion_state* résumant l'état de la discussion. Ensuite, les deux agents reprendront leur événement actuel *current_event* une fois que cette information sera partagée et que la discussion sera affichée sur l'interface graphique (front-end).

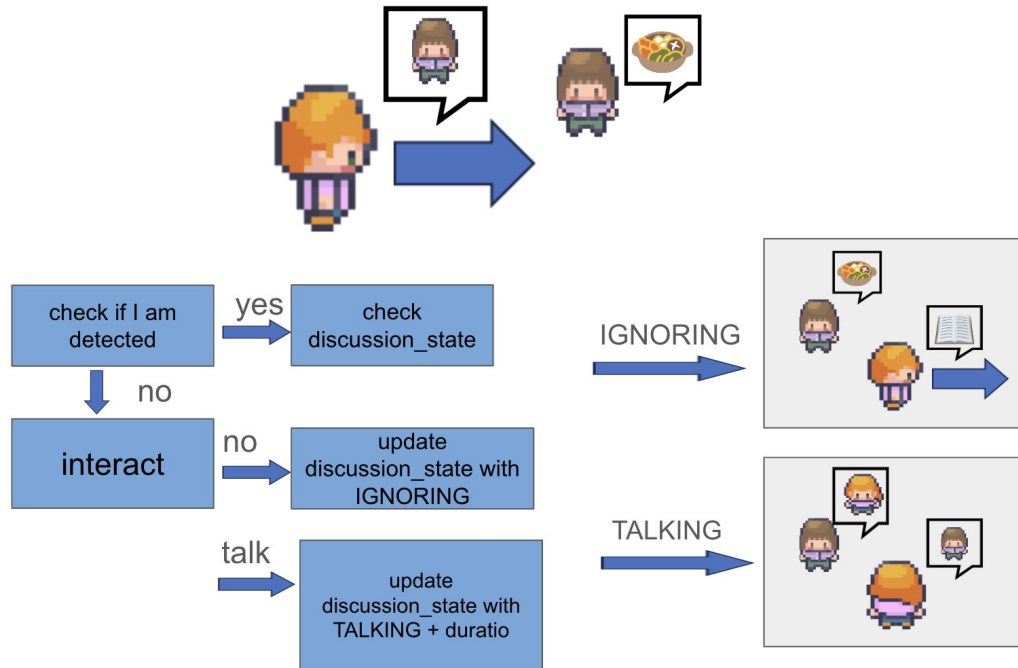


FIGURE 6 – Modélisation de l'interaction entre agents

Étant donné le coût temporel et financier élevé associé à la génération phrase par phrase d'une discussion, nous avons opté pour la création de l'intégralité de la discussion une fois que celle-ci est souhaitée par la méthode "interact". Pour ce faire, nous fournissons les informations suivantes au modèle GPT-3.5-turbo afin de générer une discussion pertinente :

1. Les sentiments de l'agent 1 à l'égard de l'agent 2, et vice versa.
2. Les tâches importantes accomplies par les deux agents dans la journée.
3. Les tâches importantes prévues par les deux agents pour le reste de la journée.

4. Un événement significatif relatif à l'événement actuel dans la mémoire épisodique des deux agents.



FIGURE 7 – Interaction entre Alice et James

2.5.2 • DÉCALAGE DU PLAN LORSQU'IL Y A UNE INTERACTION

Dans notre modèle initial, le plan initial de chaque agent est généré par le prompt décrit précédemment en début de journée. Ensuite, au début de chaque tâche, celui-ci est subdivisé en sous-tâches qui peuvent être exécutées par l'agent. Dans ce contexte, il est impossible de prédire s'il y aura une discussion ou non, ainsi que la durée de chaque discussion. Par conséquent, il est impératif de sortir de l'exécution pour modifier le plan initial afin d'ajouter une discussion.

Néanmoins, nous avons opté pour une approche itérative pour découper les tâches une par une. À chaque itération, nous prenons le plan initial et découpons la prochaine tâche en exécutant les sous-tâches dans l'ordre. Une fois que le découpage avec l'exécution est lancé, nous ne savons pas s'il y a une ou plusieurs discussions pendant l'exécution, et nous ne pouvons pas modifier la tâche et les sous-tâches qui sont en train d'être exécutées pour y ajouter la durée des discussions.

Ainsi, nous avons opté pour une modification tardive, c'est-à-dire que nous modifions le plan initial chaque fois qu'une discussion est censée être lancée, et cette modification sera validée lors du prochain découpage. Nous choisissons alors une tâche qui n'a pas encore été exécutée d'une durée suffisante, et nous réduisons sa durée exactement de la durée de la discussion. Ensuite, les tâches et les sous-tâches devant cette tâche non exécutée seront décalées de cette durée afin que la tâche en cours d'exécution dure plus comme il y a une discussion. Dans ce scénario, les discussions ne seront pas affichées sur le plan mais seront stockées en mémoire si elles dépassent un seuil (qui est bien réel car nous ne pouvons pas "planifier" une discussion dans la vie!), mais la durée de la discussion est prise en compte dans le plan afin que celui-ci corresponde aux activités de l'agent affichées sur le front-end.

2.6 RÉFLEXION

Nos agents ont maintenant la capacité de planifier leur journée, d'exécuter les tâches prévues, de communiquer entre eux et d'agir en conséquence. Cependant, notre objectif est de développer des agents qui manifestent des comportements humains, s'éloignant le plus possible des réactions robotiques, algorithmiques. Pour ce faire, nous avons créé une fonction permettant à l'agent de générer des pensées sur un événement.

La fonction "think" permet à l'agent de simuler une forme d'introspection : de prendre du recul, et de générer des idées créatives et innovantes qui sont cohérentes avec sa personnalité et son vécu. Cela confère aux agents une dimension de spontanéité, d'imprévisibilité, les faisant agir de façon plus nuancée, enrichissant une potentielle expérience de jeu. Ainsi, lorsqu'un événement est ajouté à la mémoire épisodique de l'agent et que son importance dépasse un seuil prédéfini (fixé après des tests, afin de limiter les moments de réflexion à environ une fois par heure), l'agent est invité (à l'aide, comme toujours d'un prompt et de GPT-3.5-turbo) à réfléchir sur cet événement et ses événements connexes.

... Given these elements, I would like to generate, in a sentence, a thought that #agent_name# could have, reflecting their emotional complexity, intellect, and how these aspects interact with recent events in their life and their environment. ...

Ces réflexions, sauvegardées dans la mémoire épisodique sous l'intitulé "Thought : ...", permettent aux agents d'apprendre de leurs expériences et d'ajuster leur plans futurs en adéquations avec leurs pensées antérieures, leurs désirs et leurs idées.

Voici un exemple de pensée générée par cette fonction :

John thought : "I should incorporate interval training into my morning exercises to improve my triathlon performance and make a protein-packed smoothie for a nutritious breakfast."

3

MODÉLISATION DU FRONT-END

3.1 UTILISATION DE TILED ET DE LA BIBLIOTHÈQUE PYGAME

Nous avons opté pour l'utilisation de l'application Tiled pour la conception de la carte graphique. Tiled est un éditeur de cartes open source spécialement conçu pour la création de cartes 2D destinées aux jeux ou à d'autres applications interactives.

On a aussi utilisé une bibliothèque d'images (*Tilesset*) faite par LimeZu et publiée sur le site itch.io qui est dédié à distribuer des productions pour jeux vidéo.

Dans Tiled, chaque *tile* correspond à un carré de pixels spécifiques, et il permet d'importer des ressources pour la conception. Cette méthodologie s'aligne parfaitement avec notre implémentation côté back-end : chaque *tile* représente un élément au sein de la matrice de "grid_roads". Les *tiles* indiquent également les routes où les agents peuvent se déplacer, représentées par des 1 dans la matrice, et par des 0 pour les obstacles. Cette matrice est essentielle pour la recherche de chemins pour les agents. Pour simplifier la création de la matrice, nous avons intégré une couche de *tiles* nommée "roads". Cette couche est directement importée dans le fichier JSON et transformée en matrice, facilitant ainsi le processus de génération de la matrice utilisée dans notre système.

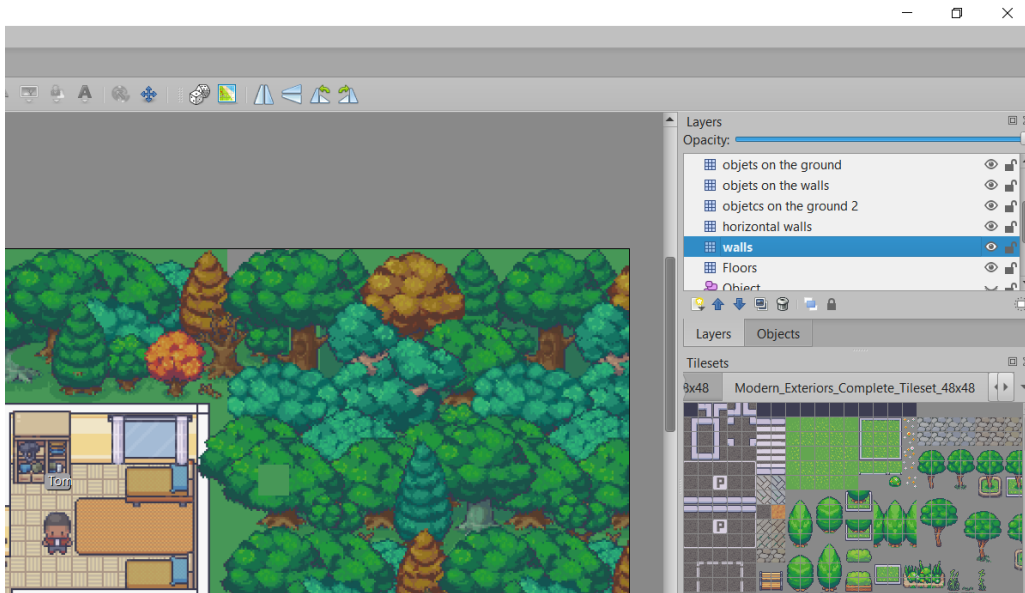


FIGURE 8 – Image extraite de Tiled

Comme mentionné dans la partie environnement, nous avons utilisé les couches d'objets de Tiled pour différencier trois types d'emplacements interactifs pour les agents : les bâtiments, les

pièces (zones) et les objets. Chaque type occupe sa propre couche, ce qui facilite l'importation directe à partir du fichier JSON. Lors de la conception, la partie graphique de la carte a été préalablement élaborée à l'aide des calques de Tiled. Il suffit de délimiter les zones interactives dans la couche d'objets, et le fichier JSON enregistre tous les noms et les coordonnées de pixels correspondant à ces zones.

Une fois cette couche importée dans la fonction "prepare_map", les coordonnées de pixels délimitant les zones sont converties en coordonnées entières dans la matrice "grid_roads" à l'aide de la fonction "get_cases". En parallèle, les noms de tous les emplacements de cette couche sont également stockés aux emplacements correspondants dans "case_details", afin de faciliter leur utilisation ultérieure lors de la recherche de chemins.

Le front-end utilise deux bibliothèques principales de Python : Pygame et Pytmx. Pygame est une bibliothèque de développement de jeux qui permet de créer des graphismes et de gérer les entrées utilisateur (comme la souris et le clavier). Pytmx est une bibliothèque qui permet de charger et de manipuler des fichiers de cartes au format *Tiled Map Editor* (TMX).

Dans ce contexte, la notion de FPS (Frames Per Second) est importante. Les FPS déterminent le nombre de fois par seconde que l'écran est rafraîchi. Cela garantit une animation fluide et réaliste.

À chaque rafraîchissement d'écran, la carte du jeu est effacée pour mettre à jour l'arrière-plan. Ensuite, en fonction des instructions provenant du back-end, tous les agents sont mis à jour sur la carte. Cela inclut généralement leur déplacement.

Étant donné que la carte peut être plus grande que l'écran sur lequel le jeu est affiché, les développeurs ont inclus la possibilité pour les utilisateurs d'ajuster la taille de l'écran en utilisant la souris et le clavier. De plus, il y a une fonctionnalité qui permet de suivre le déplacement d'un agent en cliquant sur un bouton. Cela garantit que les utilisateurs peuvent interagir avec le jeu de manière flexible et répondre à leurs besoins et envies spécifiques.

3.2 CONNEXION BACK-END -> FRONT-END

Le front-end et le back-end sont exécutés en parallèle dans des processus différents. On a donc besoin d'une liaison entre ces deux parties : Un attribut crucial de l'environnement est la "queue_of_configurations", une liste des actions que chaque agent entreprend, partagée par tous les processus (donc tous les agents et surtout le front-end). Cette caractéristique permet une liaison fondamentale entre le front-end et le back-end du système, facilitant la communication et la coordination des actions des agents dans l'environnement simulé. En effet, cela permet de générer en avance dans le back-end les différentes actions de l'agent, assurant une fluidité au niveau du front-end (puisque l'on ne peut pas contrôler la durée de génération d'une action dans le back-end).

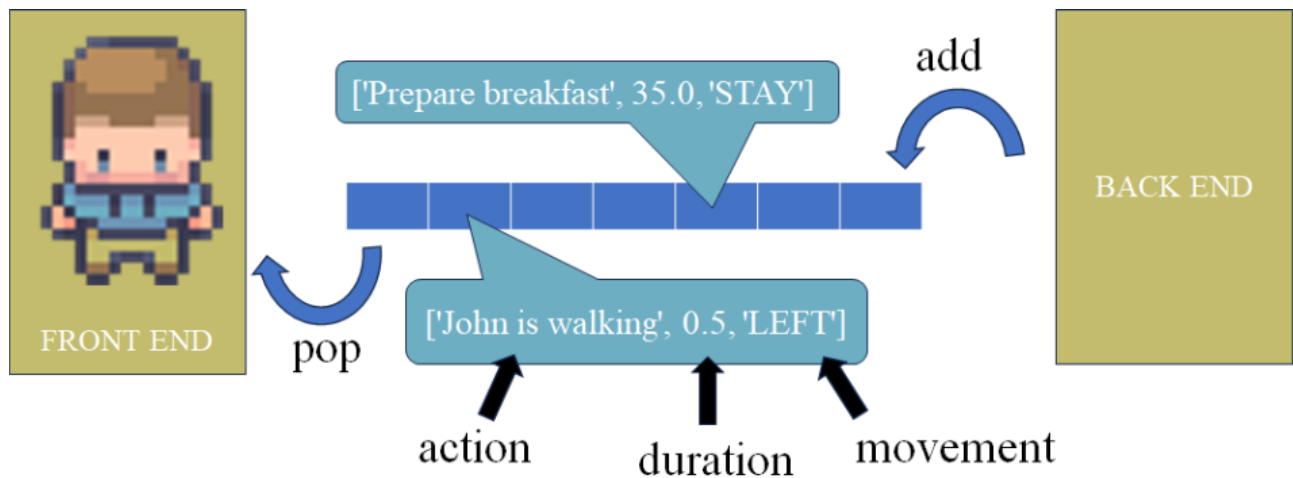


FIGURE 9 – Schéma montrant le fonctionnement de "queue_of_configurations" pour lier le front-end au back-end

En plus de cela, le front-end a accès à certains fichiers JSON : identity, episodic et chat. Cette accès permet d'afficher les informations pertinentes de chaque PNJ et de suivre sa mémoire épisodique ainsi que ses discussions.

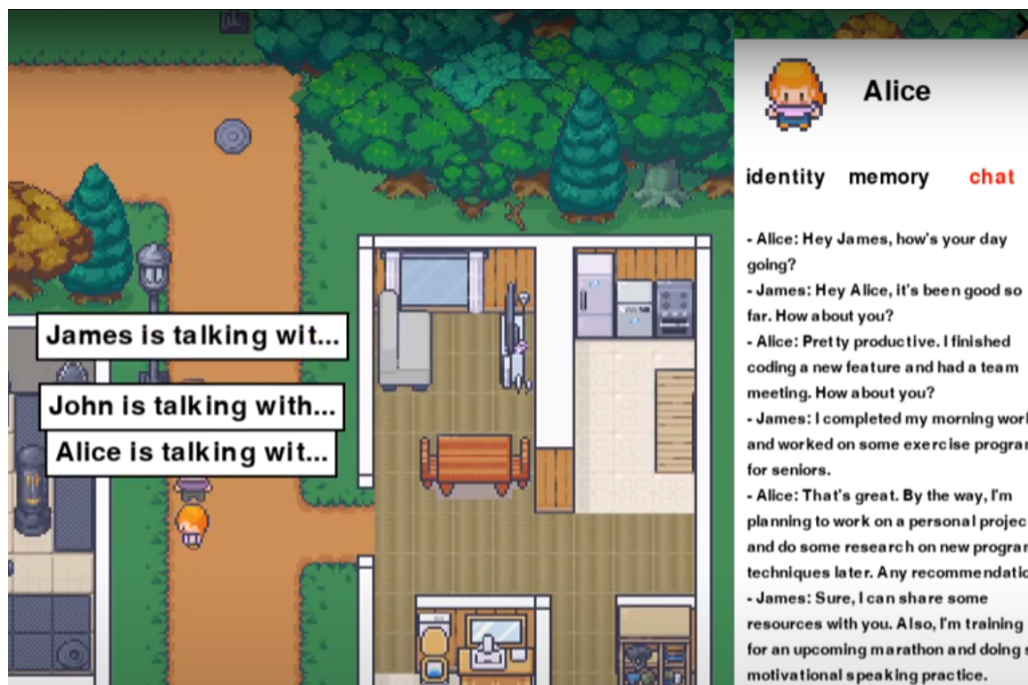


FIGURE 10 – Image qui montre la discussion entre deux PNJ

4

HISTORIQUE ET POINTS CLEFS

De septembre 2023 à avril 2024, notre projet a évolué à travers plusieurs améliorations significatives, devenant progressivement de plus en plus complexe. Initialement, nous avons mis en place un système de mémoire et de perception, donnant les bases nécessaires pour l'introduction d'un premier joueur. Peu après, le projet a franchi une étape importante avec l'ajout de deux autres joueurs, enrichissant l'expérience et la dynamique du jeu. Ensuite, nous avons concentré une grande partie de nos efforts sur la gestion des interactions entre les joueurs, un aspect crucial pour le projet. Enfin, dans un élan d'expansion et de volonté de créer un village plus réaliste, trois autres joueurs ont été intégrés, portant le total à six. Cette extension a permis d'élargir les scénarios de jeu et d'augmenter le nombre d'interactions.

Nous avons réussi à créer une première version de notre jeu, comprenant une petite carte avec un seul PNJ. Le jeu fonctionne bien dans cette configuration, cependant, nous ne pouvions pas évaluer pleinement ses performances en raison de la petite taille de la carte et du nombre limité d'actions que le PNJ pouvait entreprendre. Pour remédier à cela, nous avons rapidement décidé de créer une carte plus grande et d'ajouter deux PNJ supplémentaires.



FIGURE 11 – Première carte avec un joueur

Cependant, dès que nous avons augmenté la taille de la carte et le nombre de PNJ, nous avons rencontré rapidement des problèmes liés aux performances du jeu, notamment des baisses importantes de FPS et des problèmes liés aux prompts (prompts trop longs et réponses pas

adaptées). Après avoir résolu ces problèmes, nous avons pu obtenir une deuxième version du jeu avec une carte plus grande et trois PNJ. Cela nous a permis d'évaluer notre modélisation de manière plus approfondie et de détecter les problèmes liés à cette expansion.



FIGURE 12 – Deuxième version de la carte plus détaillée avec 3 joueurs indépendants

Tel fût, modulo quelques fonctionnalités mineures, l'état de l'art de notre projet lors du rapport intermédiaire.

Voici donc la version finale présentée dans ce projet. Cette dernière phase du projet est marquée notamment par la gestion de l'interaction, allant de la décision d'interagir à la discussion textuelle. Cette version inclue également la réflexion et l'ajout de 3 nouveaux joueurs.



FIGURE 13 – Troisième et dernière version de la carte avec 6 joueurs en interaction

5

DIFFICULTÉS RENCONTRÉES

5.1 MODÉLISATION ET IMPLÉMENTATION DE LA MÉMOIRE

Dans les sections précédentes, nous avons souligné que le mécanisme de mémoire qui guide les agents constitue la pierre angulaire de notre projet. Il est essentiel de doter ces agents d'une base de mémoire afin de les humaniser, garantissant ainsi leur conformité avec leur histoire et leur identité. La modélisation de cette mémoire doit être particulièrement astucieuse, car les exigences des LLM impliquent des prompts aussi succincts que possible pour éviter tout retard dans la réponse ou des coûts excessifs, tout en conservant la capacité de reproduire toutes les informations pertinentes de la mémoire de l'agent pour assurer la cohérence des réponses du modèle LLM. Par conséquent, la modélisation de la mémoire a constitué le principal défi de notre projet, nous obligeant à nous appuyer sur les résultats de nos recherches pour concevoir une mémoire sous forme de flux d'informations traversant des compartiments dotés de fonctionnalités spécifiques (voir la section sur Modélisation de la mémoire). Tout au long du développement du projet, nous avons continué à réévaluer notre approche de modélisation de la mémoire. Nous avons également tiré parti de notre compréhension de la vie sociale et de notre propre expérience de la mémoire pour guider nos efforts. Cette phase du projet a suscité un fort engagement de la part de tous les membres de l'équipe, chacun apportant des perspectives uniques et des solutions basées sur leurs recherches respectives. La seule exigence était de concevoir une mémoire flexible, facilement modifiable et offrant un accès aussi simple que possible en termes de temporalité et d'espace. Notre objectif était de reproduire la mémoire humaine, en assurant son intégration fluide et son évolution continue dans le cadre du projet. Cela a constitué le cadre général de notre approche pour développer et modéliser la mémoire de l'agent.

Du point de vue de l'implémentation, nous avons adopté une approche progressive en mettant en œuvre la mémoire par étapes en fonction de l'évolution du projet. Chaque compartiment de la mémoire a été implémenté au moment adéquat, en fonction de son utilisation prévue, que ce soit pour alimenter des prompts ou stocker des informations. Cette approche nous a permis de mieux comprendre notre modèle, en intégrant chaque partie au bon moment et à la bonne place. Sur le plan technique, nous avons initialement mis en œuvre la mémoire épisodique sous forme d'un graphe d'informations, avec des nœuds représentant des informations et des arêtes portant le poids de leur similarité cosinus. Cette approche reposait sur la bibliothèque Python Networkx. Cependant, nous avons ensuite opté pour une implémentation basée sur un fichier JSON, ce qui présente l'avantage de stocker également l'encodage de chaque phrase (*embedding*), évitant ainsi de devoir recalculer l'encodage avec le modèle LLM, ce qui prendrait du temps. Cette phase de l'implémentation de la mémoire s'est déroulée en plusieurs étapes, en fonction de la maturité et de la complexité du projet, nous permettant finalement d'aboutir

à une implémentation répondant à l'ensemble des exigences que nous nous étions fixées dès le début de notre travail.

5.2 LE CHOIX DES PROMPTS

La question de la gestion des prompts est un axe majeur de notre projet car permettant d'obtenir des réponses plus ou moins pertinentes selon notre besoin. En effet, chaque prompt utilisé a fait l'objet de plusieurs tests pour vérifier la pertinence des réponses tout en cherchant à en réduire la quantité d'informations fournies (explications ci-dessus).

L'enjeu repose donc sur deux facteurs. Tout d'abord la formulation, afin de formuler une demande claire, précise et efficace. Ensuite, et de plus important, le processus de sélection des informations disponibles. Voici une liste exhaustive des éléments possibles de sélections :

Personnalité (version courte et longue), état émotionnel, relation avec un autre agent (version courte et longue), l'événement actuel (sous forme d'action, de discussion ou de pensée), le plan passé et futur, la mémoire épisodique (où l'on peut extraire : un événement, ses n plus proches voisins ou ses voisins dont la similarité dépasse un seuil).

Ces deux facteurs sont à manier dans l'objectif de ne pas générer des hallucinations, c'est à dire des comportements déviants, que l'on ne souhaite pas. En effet, des réponses inadéquates de GPT-3.5-turbo peuvent survenir lors de l'utilisation du modèle pour générer des décisions d'agents ou des coefficients qui contribuent à évaluer une situation pour un agent. Ces problèmes sont parfois directement liés au modèle. Par exemple, dans le cas de la génération de localisations pour effectuer des tâches, la réponse générée peut parfois différer de celle attendue (il est arrivé que John aille se brosser les dents aux toilettes de sa voisine par exemple). De plus, il peut arriver que le modèle produise une forme de réponse inappropriée, générant parfois des caractères parasites au lieu d'un format JSON attendu.

D'une part, la compréhension des particularités du modèle GPT-3.5-turbo, sur des principes de *prompt engineering*, nous a permis de surmonter une partie de ces difficultés. Une approche actuelle pour résoudre ce genre de problème est d'ajouter des restrictions dans les prompts telles que "Don't ... / Do not include ... / The answer must be only ...", afin de clarifier les attentes et guider le modèle vers des réponses plus appropriées.

D'autre part, nous avons du faire preuve d'analyse sur la manière de "penser" de GPT-3.5-turbo et les biais pouvant être impliqués. Un exemple de ceci se retrouve dans la fonction "interact", où GPT-3.5-turbo doit fournir le nom de la personne avec laquelle l'agent souhaite interagir ou *NO* sinon. Nous avons initialement inclus dans nos prompts l'intégralité des plans passés et futurs de l'agent, ce à quoi GPT-3.5-turbo répondait systématiquement par *No*. Nous pensons que le modèle interprète ces plans comme devant être exécutés, même quand le prompt indiquait explicitement le contraire. Ainsi, nous avons opté pour la suppression dans le prompt du plan futur envisagé, réglant instantanément le problème, rendant la fonction "interact" fonctionnelle. Par ailleurs, le choix de l'agent avec lequel interagir s'est avéré (trop) souvent peu

judicieux. Pour y remédier, nous avons revu la gestion des relations au sein de notre projet, passant d'une simple liste suivant le modèle précédent à l'intégration d'une description narrative. Par exemple en ajoutant les liens de parenté, de passé partagé, etc.

5.3 EXÉCUTION EN PARALLÈLE

Dans notre projet, nous avons envisagé d'adopter une approche de parallélisme pour gérer les agents. Chaque PNJ serait géré de manière indépendante des autres dans des processus distincts, ce qui justifie l'utilisation de la bibliothèque "multiprocessing". Cette approche promet une simulation plus réaliste et permet de gagner du temps d'exécution, notamment en raison des requêtes API qui peuvent prendre du temps. Cependant, cette méthode présente des défis, notamment en ce qui concerne le contrôle des agents et leur synchronisation lors des interactions et des actions. Nous avons donc envisagé de modifier notre approche pour adopter une approche hybride entre le parallèle et le linéaire, en effectuant les requêtes API en parallèle tout en traitant les actions des PNJ de manière linéaire. Toutefois, ce changement nécessiterait un investissement important en temps et impliquerait des modifications radicales de notre code. Par conséquent, nous avons décidé de nous concentrer sur la première approche et de résoudre les problèmes de contrôle et de synchronisation des agents au fur et à mesure qu'ils se présentent.

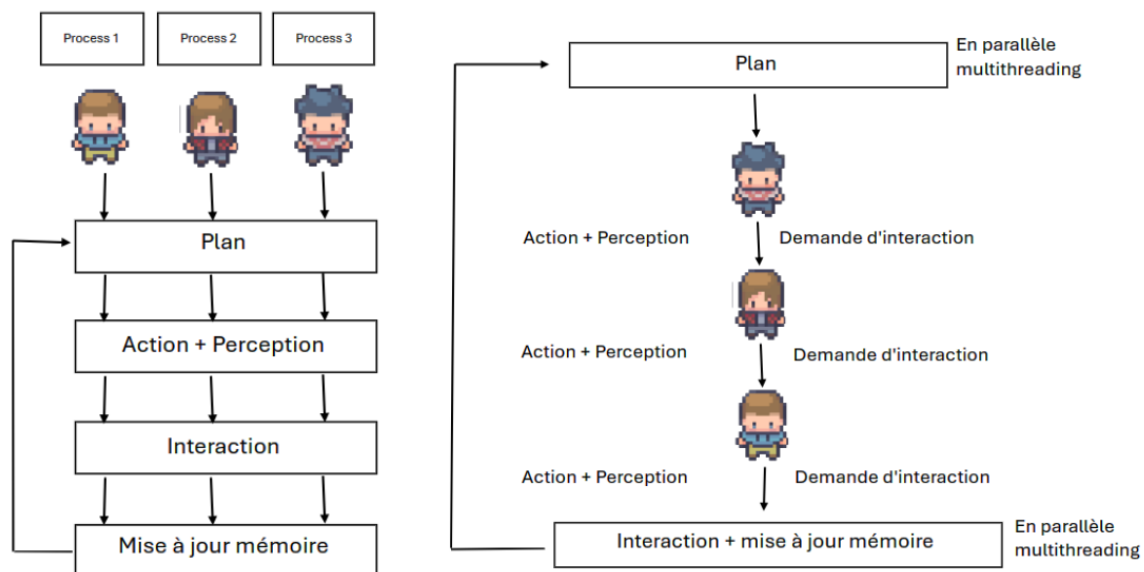


FIGURE 14 – Deux schémas qui montrent les deux approches d'exécution du jeu : à gauche l'approche parallèle (utilisé dans le projet) où chaque agent est géré dans un processus et à droite l'approche hybride qui combine entre parallélisme et linéarisation de l'exécution

5.4 GESTION DU TEMPS ET OPTIMISATION DES SCRIPTS

Étant donné que notre projet s'appuie largement sur l'implémentation de codes en Python, il était primordial d'optimiser la qualité du code et sa logique sous-jacente afin d'assurer à la fois de meilleures performances en termes de temps et d'espace, ainsi qu'une meilleure lisibilité. Cette nécessité est rapidement devenue évidente, car nous avons constaté la présence de codes redondants et peu lisibles, soulignant ainsi l'importance de convenir de formats et de conventions d'écriture. De plus, nous nous sommes fixé pour objectif de réduire au maximum la complexité temporelle, aussi insignifiante soit-elle, afin d'exercer un contrôle précis sur la dimension temporelle de notre simulation. Par conséquent, l'optimisation des scripts et leur documentation sont devenues des pratiques courantes et habituelles dans notre processus de développement.

Voici la structure de notre projet :

- **back_end/**
 - agent/
 - long_term_memory/
 - map/
 - memory/
 - prompts/
 - run_back_end.py
 - useful_functions.py
- **front_end/**
 - map_assets/
 - png/
 - tiled_map/
 - effects.py
 - game.py
 - map.py
 - person.py
 - pygame_functions.py
 - run_front_end.py
 - settings.py
- **agent.py**
- **environment.py**
- **main.py**

6

ORGANISATION DU TRAVAIL

Durant ce projet, nous avons traversé plusieurs phases d'organisation du travail, chacune issue d'une amélioration de la précédente. En effet, il s'agissait pour la plupart d'entre nous du premier projet de cette ampleur (en termes de nombre de gens impliqués et de temps alloué). Initialement, notre stratégie consistait à diviser les tâches, travailler individuellement, puis rassembler et intégrer les contributions avant de recommencer le cycle la semaine suivante.

Ce premier modèle a fonctionné pendant les premières semaines mais dès lors que le projet s'est développé, la division des tâches est devenue plus complexe en raison d'une interdépendance croissante entre les différentes parties du projet.

De plus, la coordination du travail en dehors de nos réunions hebdomadaires s'est avérée difficile. En effet, on ne travaillait pas forcément au même moment de la semaine. Ainsi, les initiatives individuelles, que ce soit en terme de structure ou de conventions, bien que nécessaires pour avancer, ont parfois mené à des redondances de travail et à une frustration chez ceux qui se sentaient exclus du progrès du projet.

Un autre défi notable a été la cohérence et la compréhension du code entre les membres de l'équipe, exacerbé par des différences linguistiques et des styles de codage variés. Pour y remédier, nous avons standardisé l'utilisation de l'anglais pour le code (plus précis pour GPT-3.5-turbo également) et exigé des commentaires clairs et complets.

Nous avons également constaté l'importance d'un processus de décision efficace. L'absence d'un "chef de projet" a parfois entraîné des débats prolongés sur des méthodes alternatives pour atteindre les mêmes objectifs, notamment lorsque deux membres de l'équipe étaient en désaccord et ne voulaient pas abandonner leurs idées. Nous nous sommes ici rendu compte de l'importance d'un décideur dans un projet de groupe afin d'être productif et efficace.

Ainsi, le projet a connu une période de mou, non pas en terme de production mais en terme d'organisation. Pour résoudre cela, nous avons imposé un type de communication : avant de travailler, nous devons annoncer sur quoi nous allons travailler, puis une fois que ce travail est chargé (*push*) sur GitHub, faire un résumé aux autres membres du groupe.

Sur les derniers mois du projet, nous avons réussi à faire fonctionner la structure de répartition des tâches décrite en début de paragraphe, en apprenant de nos erreurs : le contenu était certes centrale dans la discussion, mais le lien avec le reste du projet et le travail des autres étaient discuté sérieusement.

De plus, la complexification du projet a naturellement conduit à une spécialisation des rôles basée sur les affinités et les compétences de chaque membre de l'équipe, facilitant ainsi la collaboration et l'efficacité de notre travail collectif.

7

DEGRÉ DE RÉALISATION DES OBJECTIFS

7.1 OBJECTIFS

Objectifs Scientifiques :

1. Créer un environnement où des agents interagissent de manière similaire à des humains, sans intervention de l'utilisateur :
 - Théoriser, concevoir et améliorer un modèle informatique d'agents afin de reproduire au mieux le comportement humain.
 - Suivre une approche itérative pour améliorer la structure des agents.
 - Réaliser des tests sociologiques pour évaluer le comportement des agents.
 - Estimer les limites du modèle.
2. Réaliser une implémentation open source :
 - Mettre en place le code source de manière ouverte et accessible à d'autres développeurs et chercheurs.

Objectifs Pédagogiques :

1. Apprentissage de la programmation en Python.
2. Apprendre la structure des données (graphes, base de données) et la théorie informatique :
 - Comprendre les concepts de base en informatique, notamment la manipulation et la gestion des données.
 - Apprendre à utiliser des graphes et des bases de données pour stocker et gérer l'information.
3. Comprendre le mécanisme de l'intégration de l'IA dans les jeux :
 - Acquérir une compréhension de la manière dont l'intelligence artificielle peut être utilisée dans des environnements de jeu.
4. Acquérir des connaissances fondamentales sur le fonctionnement du cerveau humain :
 - Étudier les bases des neurosciences et comprendre comment les cerveaux humains traitent l'information et interagissent.

7.2 DEGRÉ DE RÉALISATION DES OBJECTIFS

Dans le cadre de nos objectifs scientifiques, nous avons réussi à créer un environnement dans lequel des agents interagissent de manière similaire à des humains, sans intervention de l'utilisateur. Cela a été réalisé en théorisant, en concevant et en améliorant un modèle informatique

d'agents dont le comportement se rapproche le plus possible de celui des humains. Pour ce faire, nous avons suivi une approche itérative visant à améliorer la structure des agents. Au début, nous avons créé un environnement avec seulement un agent, où il peut vivre librement. C'était notre première étape dans la modélisation du comportement des agents. Ensuite, nous avons étendu cet environnement pour inclure trois agents, comprenant un couple et un individu, leur permettant ainsi d'interagir entre eux de manière réaliste. Avec un environnement comprenant des maisons, des gymnases, des restaurants et un bureau, nous avons dû résoudre des problèmes tels que la parallélisation des agents, le partage d'informations et l'affichage de l'interface utilisateur. Lorsque trois agents pouvaient interagir parfaitement et terminer leur journée dans cet environnement, nous avons augmenté la taille du village jusqu'à six personnes, en introduisant des modèles plus complexes pour simuler le fonctionnement du cerveau humain. À ce stade, le village était devenu convivial et réaliste.

Cependant, faute de temps, nous n'avons pas pu réaliser des tests sociologiques approfondis pour évaluer le comportement des agents et estimer les limites de notre modèle, notamment pour un village de taille plus importante, comprenant une dizaine de personnages. Ce travail reste à entreprendre pour les futures promotions, si elles le souhaitent.

Par ailleurs, nous avons concrétisé une implémentation open source en publiant le code sur GitHub, rendant ainsi celui-ci ouvert et accessible à d'autres développeurs et chercheurs. Nous avons également acquis des compétences en développement front end, en utilisant des outils pertinents et en les reliant efficacement avec le back end.

En ce qui concerne nos objectifs pédagogiques, nous avons perfectionné nos compétences en programmation Python en réalisant ce projet entièrement avec ce langage. Nous avons également développé une compréhension approfondie de la structure des données, notamment des graphes, et de la théorie informatique pour les intégrer efficacement dans la gestion de la mémoire. En utilisant GPT-3.5 turbo et en surmontant ses défis, nous avons pu approfondir notre compréhension des modèles de langage et de l'ingénierie des prompts. Cette démarche nous a permis de mieux appréhender comment ces modèles peuvent être adaptés et exploités pour des applications spécifiques, telles que les jeux. Enfin, en ajoutant des éléments tels que la personnalité, les émotions, les relations et les pensées pour simuler plus précisément le comportement humain, nous avons acquis des connaissances fondamentales sur le fonctionnement du cerveau humain, en étudiant les bases des neurosciences et en comprenant comment les cerveaux humains traitent l'information et interagissent. Ces connaissances pourront être approfondies à l'avenir par des promotions ultérieures, à mesure que notre simulation deviendra de plus en plus complexe.

En résumé, nous avons globalement atteint nos objectifs, mais il reste encore des aspects à explorer et à améliorer dans les futures itérations du projet. Tout d'abord, une idée de développement serait d'utiliser la version 4 du modèle GPT pour obtenir des réponses plus précises, réduire les hallucinations et augmenter la créativité des PNJ. Cependant, le coût financier de ce modèle nécessiterait de trouver un équilibre entre l'utilisation de la version 3.5 et de la version 4, en fonction des besoins de précision pour chaque tâche. À titre indicatif, notre

projet coûte actuellement environ 20 centimes par journée de simulation. Une autre option serait d'utiliser un autre modèle de langage (LLM) tel que "Mixtral8X22b", récemment sorti, qui est open-source et très performant. Dans un monde où les LLM évoluent à une très grande vitesse, on peut envisager une amélioration drastique de la question financière et des performances dans les mois/années à venir.

Enfin, augmenter le nombre de PNJ pour ressembler davantage à un village pourrait rendre le monde du jeu plus vivant et dynamique. Les joueurs auraient ainsi l'opportunité d'interagir avec une variété de personnages ayant chacun leur propre personnalité, ce qui créerait une expérience de jeu plus diversifiée et plus intéressante.

8

CONCLUSION

Suite à notre travail approfondi sur ce sujet fascinant, qui explore la convergence entre l'intelligence artificielle et son intégration dans des idées novatrices - une tendance croissante ces dernières années - nous avons pu concevoir un projet complet. Ce projet a été rendu possible grâce au développement d'une structure de mémoire flexible et authentique, mettant en lumière la puissance des LLM lorsqu'ils sont combinés avec des mécanismes de mémoire permettant de retracer l'historique de l'agent.

La mise en place d'un plan de travail pertinent s'est avérée être un élément clé de notre réussite, car il nous a guidés à travers des phases successives, allant de la modélisation d'un mécanisme de mémoire adéquat à la conception d'un environnement où les agents peuvent être dotés de cette mémoire pour interagir avec leur environnement. Les enseignements tirés sont multiples : nous avons acquis une maîtrise de l'utilisation des LLM par le biais du prompting (ingénierie du prompt), une familiarité avec la modélisation de concepts abstraits tels que la mémoire, ainsi qu'une expertise en programmation. En outre, nous avons développé des compétences essentielles en travail d'équipe, en apprenant l'importance de la recherche, de la communication, du débat et de la gestion collective du travail pour garantir sa fluidité et son succès. Cet épanouissement continu a été déterminant dans la réussite globale de notre projet.

Notre projet, malgré son développement au sein de la chair jeu vidéo, n'en a pas la définition. En réalité, il s'agit d'une simulation dépourvue d'interaction avec les utilisateurs. Cette limitation découle principalement de notre architecture, où le back-end est plus avancé que le front-end en raison de la file d'attente des configurations. De plus, notre système de dialogue est généré en une seule fois, plutôt que phrase par phrase. Ainsi, une amélioration potentielle consisterait à revoir la connexion entre le back-end et le front-end afin de permettre aux utilisateurs d'interagir avec la simulation. Cette évolution offrirait une flexibilité accrue à la simulation, permettant aux utilisateurs de perturber le déroulement en créant de nouvelles scènes telles que l'allumage d'un incendie ou la célébration d'une fête.

Une orientation envisageable pour notre projet serait de spécialiser la simulation sur un phénomène ou un cas d'utilisation spécifique afin de reproduire ce dernier de manière originale, permettant ainsi une étude efficace. Par exemple, il serait envisageable de simuler un incendie et les réactions des personnes environnantes afin de mieux comprendre ce phénomène, ou simplement pour créer une scène réaliste dans un jeu vidéo. De plus, la prise de décision au sein d'une entreprise suscite de nombreux questionnements et pourrait être examinée en détail en dotant chaque agent de fonctions, de pouvoirs et de connaissances spécifiques au sein de l'entreprise. Cette approche simplifie l'implémentation, car le problème de simulation devient plus ciblé. Cependant, elle nécessiterait un modèle de langage naturel (LLM) efficace, voire spécifique au phénomène en question. Une solution envisageable consisterait à tirer parti de

l'évolution exponentielle des LLM et des GPU, permettant ainsi de partir de modèles de langage naturel généraux et de les adapter au cas d'utilisation spécifique par un ajustement fin (*fine tuning*).

9

BIBLIOGRAPHIE

Atkinson, R. C., & Shiffrin, R. M. (1968). Human Memory : a proposed system and its control processes. Dans Psychology of Learning and Motivation (p. 89-195). [https://doi.org/10.1016/s0079-7421\(08\)60422-3](https://doi.org/10.1016/s0079-7421(08)60422-3)

Bickmore, T., & Cassell, J. (2000). « How about this weather ? » Social Dialogue with Embodied Conversational Agents. Advances in Natural Multimodal Dialogue Systems. <https://www.aaai.org/Papers/AAAI/2000/00-04/FS00-04-002.pdf>

Devlin, J., Chang, M., Lee, K., & Toutanova, K. (2018). BERT : Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv (Cornell University). <https://arxiv.org/pdf/1810.04805v2.pdf>

Ortony, A., Clore, G. L., & Collins, A. (1988). The cognitive structure of emotions. [https://doi.org/10.1016/0010-0285\(88\)90047-9](https://doi.org/10.1016/0010-0285(88)90047-9)

Plutchik, R. (1980). A GENERAL PSYCHOEVOLUTIONARY THEORY OF EMOTION. Dans Elsevier eBooks (p. 3-33). <https://doi.org/10.1016/b978-0-12-558701-3.50007-7>

Posner, M. I. (1966). Components of skilled performance. Science, 152(3730), 1712-1718. <https://doi.org/10.1126/science.152.3730.1712>

10

ANNEXES

Vidéo de la simulation en fonctionnement :
<https://drive.google.com/file/d/1xy7uv0glthDjRVtFxO3Q6m3eC1tB2SX/view?usp=sharing>

Répertoire GitHub de notre projet :
<https://github.com/Brahim-techScope/generative-agents-with-ChatGPT>